

Atomic Methods, Thread Safety, and Race Conditions in C#

In this article, I am going to discuss **Atomic Methods, Thread Safety, and Race Conditions in C#** with Examples. Please read our previous article, where we discussed [How to Cancel Parallel Operations in C#](#) with Examples.

Atomic Methods in C#:

So far, with the Parallel Methods ([For](#), [Foreach](#), and [Invoke](#)) we have worked and performed the task independently. In the sense that they do not need external data or shared data to work. But this is not always going to be the case. Sometimes we will want to share data between threads.

An important concept to consider is the concept of **Atomic Methods**. Atomic Methods can be used comfortably in a multithreaded environment because they guarantee determinism, that is we will always obtain the same result, no matter how many threads try to execute the method simultaneously.

Characteristics of Atomic Methods in C#:

There are two fundamental characteristics of Atomic Methods in C#.

1. First, if one thread is executing an atomic method, then another thread cannot see an intermediate state that is the operation has either not started or has already been completed. But there is no intermediate state between the beginning and end.
2. Second, the operation will be completed successfully or will fail completely without making any modifications. This part is similar to database transactions where either all operations are successful or none are performed if there is at least one error.

How to Achieve Atomicity in C#?

There are several ways to achieve Atomicity in C#. The most common way is to use **locks**. Locks allow us to block other threads from executing a piece of code when the lock is activated. If you are working with collections, then another option is to use **concurrent collections**, which are specially designed to handle multithreaded scenarios. If we do not use proper mechanisms, then we will end up with unexpected results, corrupted data, or incorrect values.

Thread Safety in C#:

An important concept in a parallelism environment is thread-safe. When we say that a method is thread-safe, we are saying that we can execute this method simultaneously from multiple threads without causing any kind of error. We know that we have thread safety when the application data is not corrupted if two or more threads try to perform operations on the same data at the same time.

How to Achieve Thread Safety in C#?

What do we have to do to have a thread-safe method in C#? Well, it all depends on what we do within the method. If within the method we added an external variable. Then we could have a problem with unexpected results in that variable. Something that we can use to mitigate this is to use a synchronization mechanism like using Interlocked or using locks.

If we need to transform objects, then we can use immutable objects to avoid the problems of corrupting those objects. Ideally, we should work with pure functions. Pure functions are those that return the same value for the same arguments and do not cause secondary effects.

Race Conditions in C#:

Race conditions occur in C# when we have a variable shared by several threads and these threads want to modify the variables simultaneously. The problem with this is that depending on the order of the sequence of operations done on a variable by different threads, the value of the variable will be different. Operations are simple as increasing by one.

A variable is problematic if we do them in multithreaded scenarios on a shared variable. The reason is that even increasing by 1 a variable or adding 1 to the variable is problematic. This is because the operation is not atomic. A simple variable increment is not an atomic operation.

In fact, it is divided into three parts reading, increasing, and writing. Given the fact that we have three operations, two threads can execute them in such a way that even if we increase the value of a variable twice, only one increase takes effect.

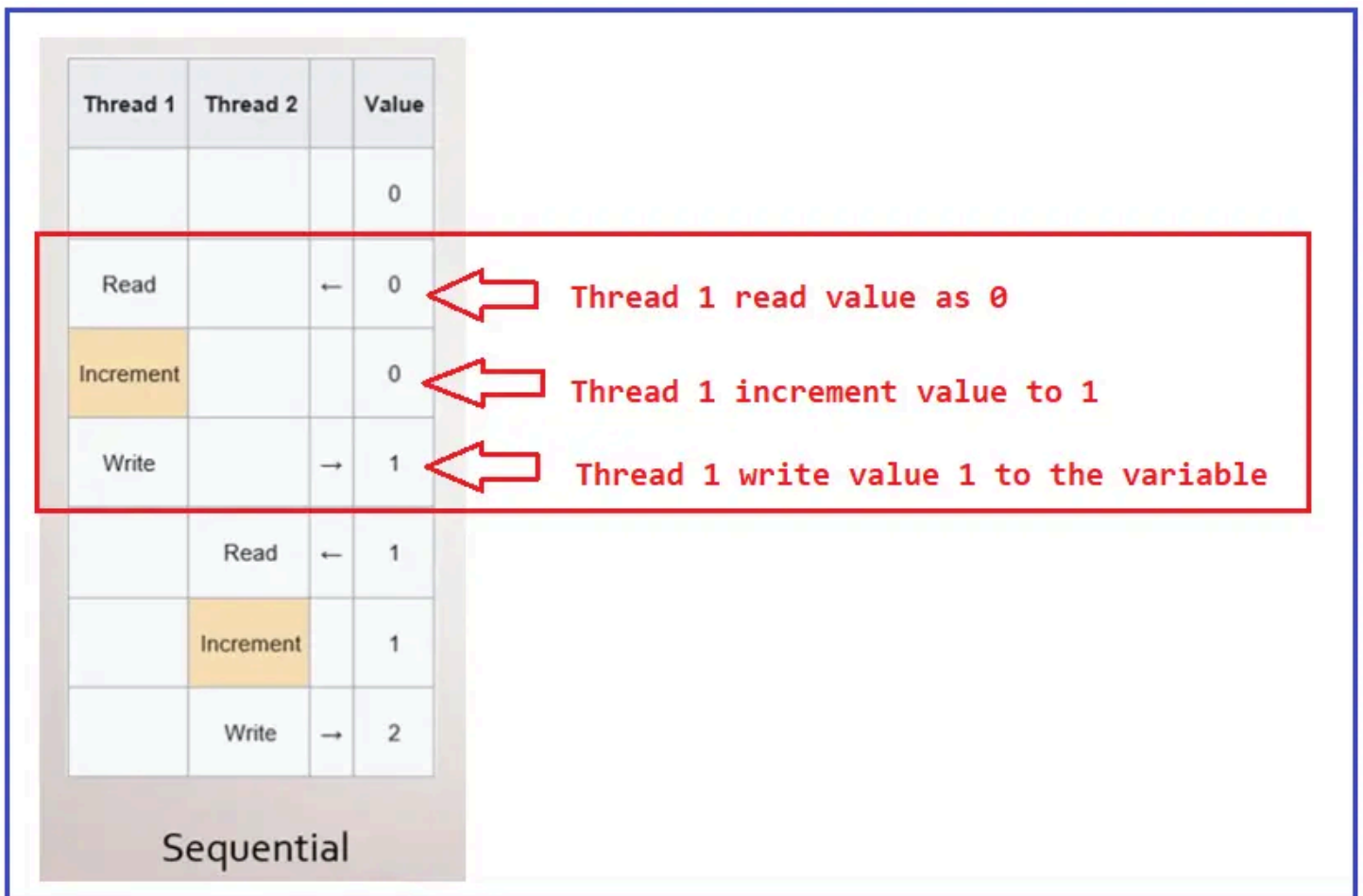
Example to Understand Race Conditions in C#:

For example, in the following table, what happens if two threads sequentially try to increment a variable? We have Thread 1 in column one and Thread 2 in column 2. And in the end, a value column that represents the value of the shared variable. For a better understanding, please have a look at the below diagram.

Thread 1	Thread 2		Value
			0
Read		←	0
Increment			0
Write		→	1
	Read	←	1
	Increment		1
	Write	→	2

Sequential

Initially, the value of the variable is zero. Then Thread 1 increments that value in memory and finally it provides that value into the variable. And then the value of the variable becomes 1. For a better understanding, please have a look at the below diagram.



Then after thread 2 reads the variable value which has now the value 1, it increments the value in memory. And finally, it writes back to the variable. And the value of the variable now is 2. For a better understanding, please have a look at the below diagram.

Thread 1	Thread 2		Value
			0
Read		←	0
Increment			0
Write		→	1
	Read	←	1
	Increment		1
	Write	→	2

← Thread 2 read the value as 1

← Thread 2 increment the value to 2

← Thread 2 write value 2 to the variable

Sequential

This is expected because of sequential execution. However, what will happen if the two threads try to update the variable simultaneously?

What happens if two threads try to update the variable simultaneously?

Well, the result could be that the final value of the variable is either 1 or 2. Let us see one possibility. Please have a look at the below diagram. Here again, we have Thread 1, Thread 2, and the value of the variable.

Thread 1	Thread 2		Value
			0
Read		←	0
	Read	←	0
Increment			0
	Increment		0
Write		→	1
	Write	→	1
Parallelism			

Now, Thread 1 and Thread 2 both read the values and so they both have the value of zero in memory. For a better understanding, please have a look at the below image.

Thread 1	Thread 2		Value
			0
Read		←	0
	Read	←	0
Increment			0
	Increment		0
Write		→	1
	Write	→	1

Parallelism

Thread 1 read the value as 0

Thread 2 read the value as 0

Thread 1 increment the value, as well as Thread 2, also increment the value and both of them increment it to 1 in memory. For a better understanding, please have a look at the below image.

Thread 1	Thread 2		Value
			0
Read		←	0
	Read	←	0
Increment		←	0
	Increment	←	0
Write		→	1
	Write	→	1

Parallelism

Once both threads increment the value to 1 in memory. Then Thread 1 writes back to variable 1 and Thread 2 also writes back to variable 1, one more time. For a better understanding, please have a look at the below image.

Thread 1	Thread 2		Value
			0
Read		←	0
	Read	←	0
Increment			0
	Increment		0
Write		→	1
	Write	→	1

Thread 1 write the variable to 1

Thread 2 write the variable to 1

Parallelism

This means that, as you can see, depending on the order of the execution of the methods, we are going to determine the value of the variable. So, even though we increase the value twice in different threads because we were in a multithreaded environment, then we had a Race condition, which means that now we do not have a deterministic operation because sometimes it could be one. Sometimes the value of the variable could be two. It all depends on chance.

How to Solve the above Problem in C#?

We can use synchronization mechanisms. The first one that we are going to use is **interlocked**. Then we will see how to use the lock to solve the race condition problem.

Interlocked in C#:

The Interlocked Class in C# allows us to perform certain operations in an atomic way, which makes this operation safe to do from different threads on the same variable. That means Interlocked class gives us a few methods that allow us to perform certain operations safely or atomically, even if the code is going to be executed by several threads simultaneously.

Example to Understand Interlocked in C#:

First, we will see the example without using Interlocked and see the problem, and then we will rewrite the same example using Interlocked and will see how interlocked solve the thread safety problem.

Please have a look at the following example. In the below example, we have declared a variable and by using the Parallel For loop we are incrementing the value. As we know Parallel.For loop uses multithreading so multiple threads trying to update (increment) the same ValueWithoutInterlocked variable. Here, as we are looping for 100000 times so we are expecting the value of the ValueWithoutInterlocked to be 100000.

```
using System;
using System.Threading.Tasks;
namespace ParallelProgrammingDemo
{
    class Program
    {
        static void Main(string[] args)
        {
            var ValueWithoutInterlocked = 0;
            Parallel.For(0, 100000, _ =>
            {
                //Incrementing the value
                ValueWithoutInterlocked++;
            });
            Console.WriteLine("Expected Result: 100000");
            Console.WriteLine($"Actual Result: {ValueWithoutInterlocked}");
            Console.ReadKey();
        }
    }
}
```

Now, run the above code multiple times and you will get different results each time, and you can also see the difference between the Actual Result and the Expected Result as shown in the below image.

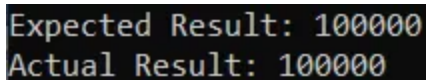
In My Case	
Expected Result: 100000 Actual Result: 47071	Run 1
Expected Result: 100000 Actual Result: 27413	Run 2
Expected Result: 100000 Actual Result: 45171	Run 3
Expected Result: 100000 Actual Result: 61572	Run 4

Example using Interlocked Class in C#:

The Interlocked Class in C# provides one static method called Increment. The Increment method increments a specified variable and stores the result, as an atomic operation. So, here we need to specify the variable with the ref keyword as shown in the below example.

```
using System;
using System.Threading;
using System.Threading.Tasks;
namespace ParallelProgrammingDemo
{
    class Program
    {
        static void Main(string[] args)
        {
            var ValueInterlocked = 0;
            Parallel.For(0, 100000, _ =>
            {
                //Incrementing the value
                Interlocked.Increment(ref ValueInterlocked);
            });
            Console.WriteLine("Expected Result: 100000");
            Console.WriteLine($"Actual Result: {ValueInterlocked}");
            Console.ReadKey();
        }
    }
}
```

Output:



```
Expected Result: 100000
Actual Result: 100000
```

As you can see in the above output image, we are getting the Actual Result as the Expected Result. So, the Interlocked Class provides atomic operations for variables that are shared by multiple threads. That means the synchronization mechanism Interlocked allows us to avoid having race conditions by making the increment operation Atomic. If you go to the definition of Interlocked class, you will see that this class provides many static methods such as Increment, Decrement, Add, Exchange, etc as shown in below image to perform atomic operations on the variable.

```

namespace System.Threading
{
    public static class Interlocked
    {
        public static long Add(ref long location1, long value);
        public static int Add(ref int location1, int value);
        public static int CompareExchange(ref int location1, int value, int comparand);
        public static object CompareExchange(ref object location1, object value, object comparand);
        public static double CompareExchange(ref double location1, double value, double comparand);
        public static float CompareExchange(ref float location1, float value, float comparand);
        public static long CompareExchange(ref long location1, long value, long comparand);
        public static IntPtr CompareExchange(ref IntPtr location1, IntPtr value, IntPtr comparand);
        public static T CompareExchange<T>(ref T location1, T value, T comparand) where T : class;
        public static long Decrement(ref long location);
        public static int Decrement(ref int location);
        public static IntPtr Exchange(ref IntPtr location1, IntPtr value);
        public static T Exchange<T>(ref T location1, T value) where T : class;
        public static float Exchange(ref float location1, float value);
        public static long Exchange(ref long location1, long value);
        public static int Exchange(ref int location1, int value);
        public static double Exchange(ref double location1, double value);
        public static object Exchange(ref object location1, object value);
        public static int Increment(ref int location);
        public static long Increment(ref long location);
        public static void MemoryBarrier();
        public static long Read(ref long location);
        public static void SpeculationBarrier();
    }
}

```

Sometimes Interlocked is not enough. Sometimes we don't multiple threads to access the critical section. We want only one thread to access the critical section. For that, we can use the lock.

Lock in C#:

Another mechanism that we can use for data editing by multiple threads simultaneously is a lock. with lock, we can have a block of code that will only be executed by one thread at a time. That is, we limit a part of our code to be sequential, even if several threads try to execute that code at the same time. We use locks when we need to perform several operations or an operation not covered by Interlocked.

Something important to take into account is that ideally what we do inside a lock block should be relatively fast. This is because the threads are blocked while waiting for the release of the lock. And if you have multiple threads blocked for a longer period of time, this can have an impact on the speed of your application.

Example to Understand the Lock in C#:

Let us rewrite the previous example using the lock. Please have a look at the below example. It is recommended to have a dedicated object for the lock. The idea is that we make locks based on objects.

```

using System;
using System.Threading.Tasks;
namespace ParallelProgrammingDemo
{
    class Program

```

```

{
    static object lockObject = new object();
    static void Main(string[] args)
    {
        var ValueWithLock = 0;
        Parallel.For(0, 100000, _ =>
        {
            lock(lockObject)
            {
                //Incrementing the value
                ValueWithLock++;
            }
        });
        Console.WriteLine("Expected Result: 100000");
        Console.WriteLine($"Actual Result: {ValueWithLock}");
        Console.ReadKey();
    }
}

```

Output:

```

Expected Result: 100000
Actual Result: 100000

```

In the next article, I am going to discuss [Interlock vs Lock in C#](#) with Examples. Here, in this article, I try to explain **Atomic Methods, Thread Safety, and Race Conditions in C#** with Examples. I hope you enjoy this Atomic Method, Thread Safety, and Race Conditions in C# with Examples.